

1.从URL到完整页面的流程

一、从输入URL到接收数据

当你在浏览器地址栏输入一个URL并按下回车键时，一系列复杂的事件在幕后悄然发生：

- 1. URL解析**：浏览器首先会解析你输入的URL，识别出其中的协议（如 `https`）、域名（如 `www.example.com`）、路径等信息。
- 2. DNS查询**：浏览器会检查本地缓存，看是否已经有该域名对应的IP地址。如果没有，它会向DNS服务器发送请求，将域名“翻译”成服务器的IP地址。这就像通过一个地址簿查找一个人的电话号码。
- 3. 建立TCP连接**：获得IP地址后，浏览器会与目标服务器建立一个TCP连接。这是一个三次握手的过程，确保双方都准备好进行数据传输。对于 `https` 协议，还需要进行TLS/SSL握手，以建立一个加密的安全通道。
- 4. 发送HTTP请求**：连接建立后，浏览器会向服务器发送一个HTTP请求。这个请求包含了请求方法（通常是 `GET`）、要访问的资源路径、以及一些请求头（Headers），用以告知服务器浏览器的类型、可接受的数据格式等信息。
- 5. 服务器处理请求并返回响应**：服务器接收到请求后，会进行处理（例如，从数据库查询数据，渲染HTML模板等），然后向浏览器返回一个HTTP响应。响应中包含了状态码（如 `200 OK` 表示成功）、响应头和响应体（Response Body），响应体通常就是我们请求的HTML文档。

二、关键渲染路径 (Critical Rendering Path)

当浏览器接收到服务器返回的HTML文档后，渲染引擎就开始工作了。这个将HTML、CSS和JavaScript转换成屏幕上像素的过程，被称为“关键渲染路径”。

- 1. 构建DOM树**：

- 浏览器从上到下解析HTML文档，将各种HTML标签（如 `<html>`，`<body>`，`<div>`）转换成一个树形结构，这就是**文档对象模型（Document Object Model, DOM）**。
- DOM是HTML文档在内存中的对象表示，它包含了文档的内容和结构。

2. 构建CSSOM树 (Building the CSSOM Tree):

- 在解析HTML的过程中，如果遇到CSS（无论是通过 `<link>` 标签引用的外部CSS文件，还是 `<style>` 标签内的内联样式），浏览器会开始解析CSS。
- 它会将CSS规则转换成一个同样是树形结构的**CSS对象模型（CSS Object Model, CSSOM）**。
- CSSOM包含了页面元素的样式信息。

3. 构建渲染树 (Render Tree Construction):

- DOM树和CSSOM树构建完成后，浏览器会将它们结合起来，创建一个**渲染树（Render Tree）**。
- 渲染树只包含需要显示在页面上的节点。例如，像 `<head>` 这样本身不可见的标签，或者通过 `display: none;` 隐藏的元素，都不会出现在渲染树中。
- 渲染树中的每个节点都包含了其在页面上的可见内容和样式信息。

4. 布局 (Layout / Reflow):

- 有了渲染树，浏览器就可以计算出每个节点在屏幕上的确切位置和大小。这个过程称为**布局（Layout）**，有时也叫**回流（Reflow）**。
- 浏览器从渲染树的根节点开始遍历，确定每个元素的几何信息（位置、尺寸）。

5. 绘制 (Painting / Rasterizing):

- 布局阶段完成后，浏览器知道了每个元素应该在屏幕的哪个位置、画多大。接下来就是**绘制（Painting）**阶段。
- 浏览器会将渲染树中的每个节点转换成屏幕上的实际像素。这个过程涉及到将文本、颜色、图像、边框、阴影等所有可见部分绘制出来。

6. 合成 (Compositing):

- 为了提高效率，浏览器可能会将页面的不同部分绘制在不同的图层（Layers）上。
- **合成（Compositing）**步骤就是将这些图层按照正确的顺序合并在一起，最终显示在屏幕上。这对于处理复杂的动画和滚动效果尤其重要，因为它可以避免对整个页面进行重新绘制。

